UNIVERSITY DATABASE MANAGEMENT SYSTEM

ASSIGNMENT 1: ER DIAGRAM

Step 1: Identification of Entities and Respective Attributes

There are six primary entities identified for this University database system. The attributes with underlines represent the Primary Keys (unique identifiers) for each entity.

- **COLLEGES**: CollegeID, Name, Address, Contact_No.
- **DEPARTMENTS**: DeptID, Dept_Name, Contact_No, HOD_Name.
- **COURSES**: CourseNo, Course_Title, Year.
- **FACULTY**: FacultyID, Name, Designation, Qualification, Address, Contact_No.
- **STUDENTS**: RollNo, Name, Year, Address, Contact_No.
- **PROGRESS_REPORT**: ReportID, Year, Grade, Rank

Step 2: Identification of Relationships with Constraints

1. MEMBER_OF Relationship (1:N)

- **Entities Involved**: **COLLEGES** and **DEPARTMENTS**.
- **Cardinality (1:N)**: One-to-Many. A single college can have many departments, but a specific department belongs to only one college.
- **Constraints**: **Total Participation**. Every department must be associated with a college to exist in the university system.

2. OFFERS Relationship (1:N)

- **Entities Involved**: **DEPARTMENTS** and **COURSES**.
- **Cardinality (1:N)**: One-to-Many. A department can offer multiple courses, but each course is typically managed by one specific department.
- **Constraints**: **Total Participation**. A course must belong to a department to be listed in the catalog.

3. TEACHES Relationship (M:N)

- **Entities Involved**: **FACULTY** and **COURSES**.
- **Cardinality (M:N)**: Many-to-Many. Per your assumptions, a lecturer can teach more than one course, and a course can be taught by more than one lecturer.
- **Relationship Attributes**: **Semester** and **Academic_Year** can be attached to this relationship to track when a specific faculty member taught a specific course.

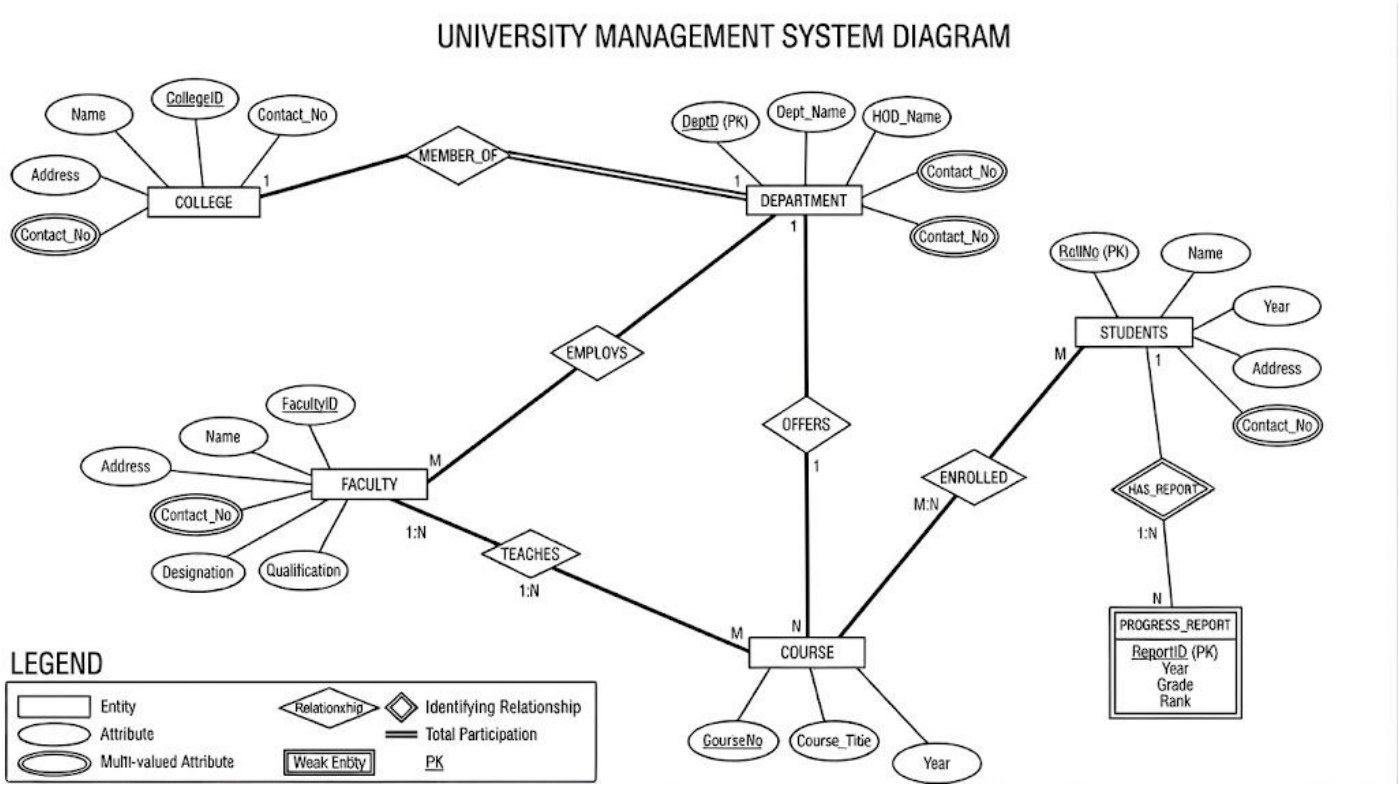
4. ENROLLED Relationship (1:N)

- **Entities Involved**: **DEPARTMENTS** and **STUDENTS**.
- **Cardinality (1:N)**: One-to-Many. A department has many students, but a student is enrolled in only one department.
- **Constraints**: **Partial Participation**. A department might exist for some time before any students are enrolled in it.

5. HAS_REPORT Relationship (1:N)

- **Entities Involved**: **STUDENTS** and **PROGRESS_REPORT**.
- **Cardinality (1:N)**: One-to-Many. A single student can have multiple progress reports (one for each year), but a report belongs to only one student.
- **Constraints**: **Partial Participation**. Per your assumption, 1st-year students will not have a Progress Report yet.

Step 3: ER Diagram:



ASSIGNMENT 2: RELATIONAL TABLES:

Step 1: Mapping ER Diagram to Tables:

The process of converting your ER diagram into a relational schema follows standard mapping rules:

1. **Strong Entities:** Each becomes a separate table with its underlined attribute as the Primary Key.
    - **COLLEGES:** (CollegeID, Name, Address, Contact_No)
    - **DEPARTMENTS:** (DeptID, Dept_Name, Contact_No, HOD_Name)
    - **COURSES:** (CourseNo, Course_Title, Year)
    - **FACULTY:** (FacultyID, Name, Designation, Qualification, Address, Contact_No)
    - **STUDENTS:** (RollNo, Name, Year, Address, Contact_No)
2. **1:N Relationships:** The Primary Key of the "One" side is added as a Foreign Key (FK) to the "Many" side.
    - **MEMBER_OF:** Adds CollegeID to the DEPARTMENTS table.
    - **OFFERS:** Adds DeptID to the COURSES table.
    - **EMPLOYS:** Adds DeptID to the FACULTY table.
3. **Weak Entities:** The Primary Key of the owner entity is added to the weak entity's table.
    - **PROGRESS_REPORT:** (ReportID, Year, Grade, Rank, *RollNo*)
4. **M:N Relationships:** These require new junction tables containing the Primary Keys of both participating entities as Foreign Keys.
    - **TEACHES:** (FacultyID, CourseNo)
    - **ENROLLED:** (RollNo, CourseNo)

Step 2: Create Tables:

```
-- 1. Colleges Table
CREATE TABLE COLLEGES (
   CollegeID INT PRIMARY KEY,
   Name VARCHAR(100),
   Address VARCHAR(255),
   Contact_No VARCHAR(15)
);
```

```sql
-- 2. Departments Table (Mapped from MEMBER_OF)
CREATE TABLE DEPARTMENTS (
    DeptID INT PRIMARY KEY,
    Dept_Name VARCHAR(100),
    Contact_No VARCHAR(15),
    HOD_Name VARCHAR(100),
    CollegeID INT,
    FOREIGN KEY (CollegeID) REFERENCES COLLEGES(CollegeID)
);

-- 3. Students Table
CREATE TABLE STUDENTS (
    RollNo VARCHAR(20) PRIMARY KEY,
    Name VARCHAR(100),
    Year INT,
    Address VARCHAR(255),
    Contact_No VARCHAR(15)
);

-- 4. Progress Report Table (Mapped from HAS_REPORT)
CREATE TABLE PROGRESS_REPORT (
    ReportID INT PRIMARY KEY,
    Year INT,
    Grade VARCHAR(2),
    Rank INT,
    RollNo VARCHAR(20),
    FOREIGN KEY (RollNo) REFERENCES STUDENTS(RollNo) ON DELETE CASCADE
);

-- 5. Courses Table (Mapped from OFFERS)
CREATE TABLE COURSES (
    CourseNo VARCHAR(20) PRIMARY KEY,
    Course_Title VARCHAR(100),
```

```
    Year INT,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENTS(DeptID)
);

-- 6. Faculty Table (Mapped from EMPLOYS)
CREATE TABLE FACULTY (
    FacultyID INT PRIMARY KEY,
    Name VARCHAR(100),
    Designation VARCHAR(50),
    Qualification VARCHAR(50),
    Address VARCHAR(255),
    Contact_No VARCHAR(15),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENTS(DeptID)
);

-- 7. Teaches Junction Table (Mapped from TEACHES M:N)
CREATE TABLE TEACHES (
    FacultyID INT,
    CourseNo VARCHAR(20),
    PRIMARY KEY (FacultyID, CourseNo),
    FOREIGN KEY (FacultyID) REFERENCES FACULTY(FacultyID),
    FOREIGN KEY (CourseNo) REFERENCES COURSES(CourseNo)
);
-- 8. Enrolled Junction Table (Mapped from ENROLLED M:N)
CREATE TABLE ENROLLED (
    RollNo VARCHAR(20),
    CourseNo VARCHAR(20),
    PRIMARY KEY (RollNo, CourseNo),
    FOREIGN KEY (RollNo) REFERENCES STUDENTS(RollNo),
    FOREIGN KEY (CourseNo) REFERENCES COURSES(CourseNo)
);
```

Step 3: Insertion of Data:

```
-- Populating Colleges
INSERT INTO COLLEGES VALUES (1, 'BPIT', 'Rohini, Delhi', '011-2704');

-- Populating Departments
INSERT INTO DEPARTMENTS VALUES (101, 'CSE', '9999999999', 'Dr. Vishal', 1);
INSERT INTO DEPARTMENTS VALUES (102, 'IT', '9811111111', 'Dr. Akansha', 1);

-- Populating Students (Including your record)
INSERT INTO STUDENTS VALUES ('09320802724', 'Kartik Goyal', 2, 'Delhi, India', '9876543210');
INSERT INTO STUDENTS VALUES ('09320802725', 'Anjali Sharma', 2, 'Dwarka, Delhi', '9866666666');

-- Populating Faculty
INSERT INTO FACULTY VALUES (201, 'Ms. Tanisha Madan', 'Assistant Professor', 'M.Tech', 'Delhi',
'9833333333', 101);
INSERT INTO FACULTY VALUES (202, 'Dr. Vishal', 'HOD', 'Ph.D', 'Rohini', '9844444444', 101);

-- Populating Courses
INSERT INTO COURSES VALUES ('CIC-206', 'Theory of Computation', 2, 101);
INSERT INTO COURSES VALUES ('CIC-210', 'Database Management', 2, 101);

-- Linking Faculty to Courses (TEACHES)
INSERT INTO TEACHES VALUES (201, 'CIC-206');
INSERT INTO TEACHES VALUES (201, 'CIC-210');

-- Linking Students to Courses (ENROLLED)
INSERT INTO ENROLLED VALUES ('09320802724', 'CIC-206');
INSERT INTO ENROLLED VALUES ('09320802724', 'CIC-210');

-- Populating Progress Reports
INSERT INTO PROGRESS_REPORT VALUES (8001, 2026, 'A', 1, '09320802724');
INSERT INTO PROGRESS_REPORT VALUES (8002, 2026, 'A+', 1, '09320802725');
```

VISUALIZING THE TABLE:

Table: Colleges

	COLLEGEID	NAME	ADDRESS	CONTACT_NO
1	1	BPIT	Rohini, Delhi	011-2704

Table: DEPARTMENTS

	DEPTID	DEPT_NAME	CONTACT_NO	HOD_NAME	COLLEGEID
1	101	CSE	9999999999	Dr. Vishal	1
2	102	IT	9811111111	Dr. Akansha	1

Table: Students

	ROLLNO	NAME	YEAR	ADDRESS	CONTACT_NO
1	09320802724	Kartik Goyal	2	Delhi, India	9876543210
2	09320802725	Anjali Sharma	2	Dwarka, Delhi	9866666666
3	09320803124	Siddharth Mehra	3	Pitampura, Delhi	9911223344
4	09320802824	Ishita Kapoor	1	Janakpuri, Delhi	9955667788
5	09320802728	Aryan Malhotra	4	Noida, Sector 62	9811224455
6	09320802730	Riya Sen	2	Gurgaon, Phase 3	9711335577
7	09320803126	Kabir Thapar	2	Vasant Kunj, Delhi	9611446688

Table: Progress_Report

	REPORTID	YEAR	GRADE	RANK	ROLLNO
1	8001	2026	A	1	09320802724
2	8002	2026	A+	1	09320802725

Table: COURSES

	COURSENO	COURSE_TITLE	YEAR	DEPTID
1	CIC-206	Theory of Computati	2	101
2	CIC-210	Database Manageme	2	101

Table: Faculty

	FACULTYID	NAME	DESIGNATION	QUALIFICATION	ADDRESS	CONTACT_NO	DEPTID
1	201	Ms. Tanisha Madan	Assistant Professor	M.Tech	Delhi	9833333333	101
2	202	Dr. Vishal	HOD	Ph.D	Rohini	9844444444	101

Table: Teaches

	FACULTYID	COURSENO
1	201	CIC-206
2	201	CIC-210

Table: Enrolled

	ROLLNO	COURSENO
1	09320802724	CIC-206
2	09320802724	CIC-210

Step 4: List Queries

1. Search Student by Roll Number:

This query retrieves the full profile of a specific student using the Primary Key.

SELECT * FROM STUDENTS WHERE RollNo = '09320802724';

	ROLLNO	NAME	YEAR	ADDRESS	CONTACT_NO
1	09320802724	Kartik Goyal	2	Delhi, India	9876543210

2. Search Courses by Department:

This query joins the COURSES and DEPARTMENTS tables to list all subjects offered by 'CSE'.

SELECT C.CourseNo, C.Course_Title, C.Year

FROM COURSES C

JOIN DEPARTMENTS D ON C.DeptID = D.DeptID

WHERE D.Dept_Name = 'CSE';

	COURSENO	COURSE_TITLE	YEAR
1	CIC-206	Theory of Computati	2
2	CIC-210	Database Manageme	2

3. Get Academic Progress for a Specific Student

This query joins the STUDENTS and PROGRESS_REPORT tables to view performance records.

```
SELECT S.Name, P.Grade, P.Rank, P.Year
FROM STUDENTS S
JOIN PROGRESS_REPORT P ON S.RollNo = P.RollNo
WHERE S.RollNo = '09320802724';
```

	NAME	GRADE	RANK	YEAR
1	Kartik Goyal	A	1	2026

4. List Faculty Assignments

This query uses the TEACHES junction table to show which professor is assigned to which subject.

```
SELECT F.Name AS Faculty_Name, C.Course_Title
FROM FACULTY F
JOIN TEACHES T ON F.FacultyID = T.FacultyID
JOIN COURSES C ON T.CourseNo = C.CourseNo;
```

	FACULTY_NAME	COURSE_TITLE
1	Ms. Tanisha Madan	Theory of Computati
2	Ms. Tanisha Madan	Database Manageme

5. Count Students Enrolled per Course

An aggregate query to determine the popularity/strength of each course.

```
SELECT C.Course_Title, COUNT(E.RollNo) AS Student_Count
FROM COURSES C
LEFT JOIN ENROLLED E ON C.CourseNo = E.CourseNo
GROUP BY C.Course_Title;
```

	COURSE_TITLE	STUDENT_COUNT
1	Theory of Computati	1
2	Database Manageme	1

6. Find Departments under a Specific College

This query identifies the organizational hierarchy within the institution.

```
SELECT D.Dept_Name, D.HOD_Name
FROM DEPARTMENTS D
JOIN COLLEGES C ON D.CollegeID = C.CollegeID
WHERE C.Name = 'BPIT';
```

	DEPT_NAME	HOD_NAME
1	CSE	Dr. Vishal
2	IT	Dr. Akansha

Conclusion:

This concludes the University Management System project. The implementation successfully maps the complex relationships from the ER diagram into a functional relational database, ensuring data integrity through Primary and Foreign Keys.